Übungsblatt 8: KI-Beschleuniger & Edge AI

Mikrocomputertechnik 3 · Datenfluss, Vektorlängen und Speicherplanung auf Papier

Musterlösung

Ziel und Arbeitsweise

Die sechs Pflichtaufgaben sind vollständig auf Papier lösbar. Geben Sie Rechenweg, Einheiten und Zählgrenze an. Alle Architekturzeiten und Speicherbudgets sind Lehrannahmen.

Bearbeitungszeit: etwa 60–75 Minuten selbstständig. **Orientierungspunkte:** 32. Im Unterricht: Aufgaben 1–2 (10 min), Aufgabe 3 (15 min), Aufgabe 4 (15 min), Aufgaben 5–6 (15 min). Lösungsbesprechungen sind in die Vorlesungsphasen eingeplant.

MAC heißt Multiply-Accumulate und zählt hier zwei Operationen. Int8: 1 B, Int32/Float32: 4 B je Element. KiB = 1024 B, MB = 10^6 B.

Aufgabe 1: Dense-Schicht und Datenbudget (5 Punkte)

Eine Dense-Schicht, vollständig verbundene Schicht, berechnet $y = Wx + b$. Eine anschließende Aktivierungsfunktion ist ausgeblendet.

$$W = \begin{bmatrix} 1 & 2 & 3 \\ 0 & -1 & 4 \end{bmatrix}, \quad x = \begin{bmatrix} 2 \\ 1 \\ 3 \end{bmatrix}, \quad b = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

1. Berechnen Sie beide Ausgänge einschließlich Zwischenrechnungen. (2 P)
2. Zählen Sie trainierbare Parameter und produktbezogene MACs. Warum sind diese Zahlen verschieden? (1 P)
3. Gewichte und Eingaben liegen als Int8, Biaswerte als Int32 vor. Wie viele gespeicherte Byte benötigen W, b und x zusammen? Wie verändert sich diese Summe, wenn zwei Int8-Ausgänge hinzukommen? Ist dies bereits die Größe der Modelldatei oder der Arena? (2 P)

Musterlösung 1

1. $y_0 = 1 \cdot 2 + 2 \cdot 1 + 3 \cdot 3 + 1 = 2 + 2 + 9 + 1 = 14$. $y_1 = 0 \cdot 2 + (-1) \cdot 1 + 4 \cdot 3 - 2 = 0 - 1 + 12 - 2 = 9$. Also $y = [14, 9]^T$. (je 1 P)
2. Sechs Gewichte plus zwei Biaswerte ergeben acht Parameter. Die sechs Produkte ergeben sechs produktbezogene MACs. Bias ist ein Parameter, wird aber nicht mit einem Eingang multipliziert. (1 P)
3. W: 6 B, b: $2 \cdot 4 = 8$ B, x: 3 B. Zusammen 17 B, mit den zwei Ausgängen 19 B. (1 P) Diese Bilanz enthält keine Strukturmetadaten, Skalen, Ausrichtung, temporären Kernelpuffer oder Laufzeitverwaltung. Sie ist weder vollständige Modelldatei noch Arena. (1 P)

Die Null im Gewichtsfeld zählt in dieser dichten Ausführung als gespeicherter Wert und als Produkt. Ein besonderer Sparse-Kernel wurde nicht angenommen.

Aufgabe 2: Roofline und Wiederverwendung (5 Punkte)

Lehrmodell R betrachtet zunächst nur Operandenverkehr vom externen Speicher zum lokalen Puffer. Jede MAC zählt zwei Operationen, jeder Operand vier Byte. Bias und Ausgabeschreiben sind zunächst ausgeschlossen. Rechenpeak $P_{\text{peak}} = 8$ Ops/Takt, Bandbreite $B = 8$ Byte/Takt. Roofline ist das Modell $P \leq \min(P_{\text{peak}}, BI)$, I die Intensität in Ops/Byte.

1. Berechnen Sie die ideale Obergrenze für $I = 0{,}25$ und $I = 0{,}5$. Wo liegt der Knick, und welche Grenze ist an beiden Punkten enger? (2 P)
2. Ein 2×2-Produkt $C = AB$ benötigt acht MACs. Naiv lädt jede MAC beide Operanden neu. Mit lokalem Halten wird jeder der acht verschiedenen Eingangswerte einmal geladen. Berechnen Sie beide Eingangsbytezahlen und Intensitäten. Nun werden zusätzlich vier Ausgaben zu je vier Byte geschrieben, ohne vorher ein altes C zu lesen. Wie ändern sich Bytezahlen und Intensitäten? (2 P)
3. Der Rechenpeak wird auf 16 Ops/Takt erhöht, die Bandbreite bleibt 8 Byte/Takt. Wird bei der ursprünglichen Eingangsintensität 0,25 die Grenze größer? Begründen Sie. (1 P)

Musterlösung 2

1. $\min(8, 8 \cdot 0{,}25) = 2$ und $\min(8, 8 \cdot 0{,}5) = 4$ Ops/Takt. Knick $I = 8/8 = 1$ Op/Byte. Beide Punkte liegen darunter; die Bandbreitengrenze ist enger. Dies sind Obergrenzen, keine Messwerte. (2 P)
2. Acht MACs ergeben sechzehn Ops. Naiv: $8 \cdot 2 \cdot 4 = 64$ B, $I = 16/64 = 0{,}25$. Lokal gehalten: $8 \cdot 4 = 32$ B, $I = 16/32 = 0{,}5$. (1 P) Zusätzlich $4 \cdot 4 = 16$ Ausgabebytes ergeben 80 beziehungsweise 48 B. Intensitäten $16/80 = 0{,}2$ und $16/48 = 1/3 \approx 0{,}333$ Ops/Byte. (1 P)
3. Nein: $\min(16, 8 \cdot 0{,}25) = 2$ Ops/Takt. Zusätzliche Rechenkapazität ändert die unveränderte Versorgung nicht. (1 P)

Die Intensitäten mit Ausgaben verwenden eine erweiterte Zählbasis. Die eingangsbezogene Roofline darf nicht ohne eine neue Bandbreitenannahme für beide Transferrichtungen als dieselbe Messgrenze ausgegeben werden.

Aufgabe 3: Vektorlänge und Zeitmodell (6 Punkte)

RVV bedeutet RISC-V Vector Extension. VLEN ist die Registerbreite in Bit, SEW die Elementbreite, LMUL die Registergruppengröße und VLMAX die Elementkapazität. AVL ist die angeforderte Restzahl, VL die aktive zurückgegebene Länge.

Es gelten die RVV-1.0-Regeln für eine unterstützte Konfiguration: Bei AVL $\leq$ VLMAX ist VL exakt AVL, bei AVL $\geq$ 2VLMAX exakt VLMAX, dazwischen ein Integer von $\lceil \text{AVL}/2 \rceil$ bis VLMAX.

1. VLEN = 128 Bit, SEW = 32 Bit, LMUL = 1. Bestimmen Sie VLMAX. Folgt daraus dieselbe Zahl physischer Rechen-Lanes? (1 P)
2. Separates Beispiel: VLMAX = 32, AVL = 40. Prüfen Sie die Folgen $32+8$, $24+16$, $20+20$, $16+24$. Geben Sie für $24+16$ beide Zeigerinkremente bei Vier-Byte-Elementen an. (2 P)
3. Wie viele Iterationen verarbeitet ein Array mit 1000 Elementen bei dieser Kapazität? Begründen Sie auch die letzten Abschnitte. (1 P)
4. Ein eigenes Chaining-Modell startet vier unabhängige Multiplikationen in Takten eins bis vier. Latenz zwei, Ergebnis jeweils am Taktende. Die zugehörigen unabhängigen Additionen haben Latenz eins und Startabstand eins. Mit Weitergabe darf jede Addition im Takt nach ihrem Produkt beginnen; ohne Weitergabe erst nach Fertigstellung aller vier Produkte. Wann endet das letzte Ergebnis in beiden Fällen? Ist das eine RVV-Zeitgarantie? (2 P)

Musterlösung 3

1. VLMAX = LMUL $\cdot$ VLEN/SEW $= 1 \cdot 128/32 = 4$ Elemente. Physische Rechen-Lanes und ihre Startabstände sind eine Implementierungsfrage. (1 P)
2. Für AVL vierzig muss die erste VL zwischen zwanzig und zweiunddreißig liegen. $32+8$, $24+16$ und $20+20$ sind erlaubt; $16+24$ beginnt unzulässig unter zwanzig. (1 P) Bei $24+16$ sind die Inkremente $24 \cdot 4 = 96$ und $16 \cdot 4 = 64$ Byte. (1 P)
3. Dreißig erzwungene volle Iterationen bearbeiten 960 Elemente. Der Rest vierzig braucht zwei zulässige Abschnitte. Insgesamt zweiunddreißig Iterationen. (1 P)
4. Produkte enden in zwei bis fünf. Mit Chaining, elementweiser Weitergabe, starten und enden Additionen in drei bis sechs. Ohne Chaining in sechs bis neun. Letztes Ergebnis: Ende sechs beziehungsweise neun. (1 P) Dies folgt aus dem gegebenen Modell, nicht aus der Befehlssatzspezifikation. Es sind in beiden Fällen vier Multiplikationen und vier separate Additionen. (1 P)

Aufgabe 4: Systolische Teilsummen (6 Punkte)

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}, \quad C = AB$$

Vier Zellen C_{ij} halten ihre Ausgabesumme lokal (Output-Stationary). Anfangs sind alle Summen null. Eine gültige MAC wird im aktuellen Takt fertig. A wandert je Takt eine Zelle rechts, B eine unten. Indizes i, j, k beginnen bei null. A_{ik} wird links in Takt $1 + k + i$ eingespeist, B_{kj} oben in Takt $1 + k + j$. Vorladen und Auslesen sind nicht Teil dieses Zeitfensters.

1. Tragen Sie die vier Teilsummen nach jedem Takt ein. Markieren Sie deren letzte gültige MAC. (3 P)
2. Welche beiden Paare erreichen C_{11} in welchen Takten? Warum ist die Summe nicht schon in Takt zwei fertig? (1 P)
3. Berechnen Sie die mittlere Auslastung über vier Takte im 2×2-Array. Dasselbe Tile läuft nun isoliert in vier Zellen eines 4×4-Arrays, alle anderen bleiben leer. Berechnen Sie zeitliche Auslastung und räumliche Belegung. Ist interne Fertigstellung bereits Readout? (2 P)

Taktende	C_{00}	C_{01}	C_{10}	C_{11}
1				
2				
3				
4				

Musterlösung 4

Taktende	C_{00}	C_{01}	C_{10}	C_{11}	gültige MACs
1	5	0	0	0	1
2	19 fertig	6	15	0	3
3	19	22 fertig	43 fertig	18	3
4	19	22	43	50 fertig	1

1. C_{00}: $1 \cdot 5 = 5$, dann $+2 \cdot 7 = 19$. C_{01}: $1 \cdot 6 = 6$, dann $+2 \cdot 8 = 22$. C_{10}: $3 \cdot 5 = 15$, dann $+4 \cdot 7 = 43$. C_{11}: $3 \cdot 6 = 18$, dann $+4 \cdot 8 = 50$. Letzte MACs in zwei, drei, drei und vier. (3 P)
2. In Zelle (i, j) treffen passende Operanden in $1 + k + i + j$ ein. C_{11} erhält drei und sechs in Takt drei, vier und acht in vier. Zeilen- und Spaltenversatz verhindern einen früheren Start. (1 P)
3. Acht MACs bei sechzehn Zell-Takten ergeben $8/16 = 50\,\%$. Im größeren Array: $8/(16 \cdot 4) = 12{,}5\,\%$ zeitlich, $4/16 = 25\,\%$ räumlich. (1 P) Fertige lokale Summen benötigen weiterhin einen Ausleseweg; dessen Zeit wurde nicht angenommen. (1 P)

Aufgabe 5: Quantisierung und Zahlenformate (6 Punkte)

Int8 hat Bereich $[-128, 127]$. Wir verwenden $q = \text{clip}(\text{round}(r/s) + z)$ und $\hat{r} = s(q - z)$. $s > 0$ ist die Skala, z der ganzzahlige Nullpunkt. Rundung: nächster Integer, exakte Halbfälle weg von null.

1. Mit $s = 0{,}1$, $z = 0$: Bestimmen Sie Code, Rückwert und absoluten Fehler für $r = 1{,}24$ und $r = 20$. Welche Fehlerart liegt jeweils vor? (2 P)
2. Mit $z = -3$: Welcher Code repräsentiert reale Null? Begründen Sie. (1 P)
3. Separates Skalarprodukt: $q_x = [12, -4]$, $z_x = -3$, $s_x = 0{,}1$, $q_w = [2, 3]$, $z_w = 0$, $s_w = 0{,}2$. Biascode $q_b = 5$ hat Skala $s_x s_w$ und Nullpunkt null. Bestimmen Sie die korrigierte Integerakkumulation und den realen Rückwert. Quantisieren Sie diesen mit $s_y = 0{,}05$, $z_y = -2$ erneut. (2 P)
4. 250 000 reine Gewichte wechseln von Float32 zu Int8. Berechnen Sie beide Bytezahlen und die prozentuale Ersparnis. Gilt sie auch für komplette Modelldatei und Arena? (1 P)

Musterlösung 5

1. $1{,}24/0{,}1 = 12{,}4 \to 12$, Rückwert 1,2, Fehler 0,04: Rundung. $20/0{,}1 = 200 \to 127$, Rückwert 12,7, Fehler 7,3: Begrenzung. Die halbe-Schritt-Fehlergrenze gilt im zweiten Fall nicht. (je 1 P)
2. $q = z = -3$, denn $s((-3) - (-3)) = 0$. (1 P)
3. Korrigierte Eingaben 15 und -1. $a = 15{\cdot}2 + (-1){\cdot}3 + 5 = 30 - 3 + 5 = 32$. $\hat{r}_y = (0{,}1{\cdot}0{,}2){\cdot}32 = 0{,}64$. (1 P) $q_y = \text{round}(0{,}64/0{,}05) - 2 = 13 - 2 = 11$. Rückwert $0{,}05(11 + 2) = 0{,}65$, Fehler 0,01. (1 P)
4. Float32: 1 000 000 B, Int8: 250 000 B. Ersparnis $1 - 1/4 = 75\,\%$ nur für Gewichtbytes. Bias, Metadaten und Arbeitsspeicher folgen anderen Regeln. (1 P)

Skalen und Nullpunkte sind Bestandteil des Tensorvertrags. Breite Akkumulation verhindert Informationsverlust durch zu schmale Zwischenergebnisse; sie ersetzt nicht die Prüfung auf einen möglichen Bereichsüberlauf.

Aufgabe 6: Firmware als Gesamtsystem (4 Punkte)

CPU bedeutet Central Processing Unit, Prozessor. SRAM ist Static Random Access Memory, statischer Arbeitsspeicher. Flash ist nichtflüchtiger Speicher, die Arena ein Tensor-Arbeitsbereich.

1. Fiktive Plattform: 128 KiB Flash, 32 KiB SRAM. Firmware 40 KiB, vollständige Modelldatei 24 KiB liegen im Flash. SRAM: Arena 12 KiB, Float-Quellfenster 1,5 KiB, Stackreserve 6 KiB, übrige Daten 4 KiB. Der Int8-Eingabetensor liegt schon in der Arena. Bestimmen Sie beide freien Restbudgets. (1 P)
2. Das Fenster enthält 128 Zeitpunkte mit je drei Achsen. Bestimmen Sie Elementzahl, Float32-Quellbytes und Int8-Zielbytes. Warum liefert `sizeof` eines Zeigers nicht diese Quellgröße? (1 P)
3. Eine Setup-Funktion erzeugt Arena und Interpreter als lokale Objekte und kehrt zurück. Eine spätere Run-Funktion verwendet sie weiter. Zusätzlich liest sie nach einem fehlgeschlagenen `Invoke()` den alten Ausgang als neue Wahrscheinlichkeit. Nennen Sie beide Fehler und die nötigen Korrekturen. (1 P)
4. Serielle Lehrzeiten: CPU-Pfad Vorverarbeitung 2 ms, Dense-Rechnung 10 ms, übrige Operatoren 3 ms, Nachverarbeitung 1 ms. Beschleunigerpfad: zusätzlich Kopie 1 ms, Dense nur 1 ms, alle übrigen Phasen unverändert. Bestimmen Sie Gesamtzeiten, Teilfaktor und Gesamtfaktor. (1 P)

Musterlösung 6

1. Flash: $128-40-24 = 64$ KiB frei. SRAM: $32-12-1{,}5-6-4 = 8{,}5$ KiB frei. Int8-Zieltensor nicht doppelt zählen. Arena ist ein separates Arbeitsbudget. (1 P)
2. $128 \cdot 3 = 384$ Elemente, Quelle $384 \cdot 4 = 1536$ B, Ziel 384 B. Ein Zeiger enthält eine Adresse; seine Größe zählt weder Elemente noch Pufferbytes. (1 P)
3. Die lokalen Objekte leben nach dem Funktionsende nicht weiter. Modell, Interpreter und Arena müssen über alle Nutzungsphasen gültig bleiben. Nach einem Fehler darf keine neue Entscheidung aus alten Ausgangsbytes entstehen: Status prüfen, Fehler behandeln, Ausgabe nur nach Erfolg gemäß Vertrag lesen. Eine Wahrscheinlichkeit muss außerdem vom Modell als solche definiert sein. (1 P)
4. CPU: $2 + 10 + 3 + 1 = 16$ ms. Beschleuniger: $2 + 1 + 1 + 3 + 1 = 8$ ms. Dense-Teilfaktor $10/1 = 10$, Gesamtfaktor $16/8 = 2$. (1 P)

Ein funktionaler Renode-Lauf wäre ohne validiertes Zeitmodell kein Nachweis dieser Hardwarezeiten oder einer Energieersparnis.

Optional: Zahlenbeispiele auf dem Host prüfen

Die Datei `src/lab08_dense.c` enthält ein vollständiges C-Programm. Sie prüft die Dense-Ausgänge, Matrixprodukte, Quantisierung und Fehlerfälle. Sie benötigt weder Modellblob noch TensorFlow Micro oder Renode. Vom Repository-Hauptordner aus:

```
gcc -std=c11 -O2 -Wall -Wextra -Werror -pedantic \
    labs/src/lab08_dense.c -lm -o /tmp/lab08_dense
/tmp/lab08_dense
```

Erwartete Ausgabe:

```
Dense: 14 9
Matrix: 19 22 43 50
Quantisierung: 12 127 -128
Alle Prueffaelle bestanden.
```

1. Lesen Sie zuerst die Verträge der beiden Hilfsfunktionen. Warum wird vor dem schmalen Cast begrenzt beziehungsweise der Bereich geprüft?
2. Vergleichen Sie die zurückgerechneten Werte mit Ihren Papierlösungen.
3. Prüfen Sie optional mit `-fsanitize=address,undefined`, `-O1` und `-g`. Welche Fehlerarten kann dieser Lauf prüfen, und welche Systemaussagen fehlen weiterhin?

Hinweise zur optionalen Aufgabe

`dense_2x3` rechnet eine feste Zwei-mal-drei-Schicht mit Int8-Operanden, Int32-Bias und Nullpunkten null. Sie akkumuliert in Int64 und prüft vor Int32-Ausgabe, ob alle Ergebnisse passen. Ausgaben bleiben bei Fehler unverändert. Das ist kein vollständiger quantisierter Netzwerkoperator.

`quantize_i8` verlangt endlichen realen Wert, endliche positive Skala, Int8-Nullpunkt und einen gültigen Ausgabepointer. Sie rundet nach der Lehrregel und begrenzt vor dem Integercast. Auch ein durch Division entstandener unendlicher Zwischenwert wird am Bereichsrand begrenzt.

Ein Sanitizer prüft im durchlaufenen Programm unter anderem ungültige Speicherzugriffe und undefinierte Integeroperationen. Er beweist weder Modellgüte noch NPU-Geschwindigkeit oder RISC-V-Zykluszahlen. RISC-V GCC wäre eine eigene Zielübersetzung; Renode bräuchte passende Firmware, Plattform und für Zeitangaben ein validiertes Zeitmodell.